ГУАП

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Ассистент		Д.С. Лисюков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ № 8

Знакомство с функционалом сервиса Apache Airflow

по курсу: Облачные технологии и сервисы

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4417		М. П. Курбатов
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Получить практические навыки развёртывания Apache Airflow 2.9.2 на персональном компьютере с помощью Docker Compose и ознакомиться с назначением основных компонентов сервиса.

2 Основные сведения

Apache Airflow используется для разработки, планирования и мониторинга рабочих процессов. DAG представляет собой направленный ациклический граф задач и их зависимостей. Оператор задаёт действие задачи, scheduler определяет готовность задач к запуску, а executor организует их выполнение. CeleryExecutor использует очередь сообщений и отдельные процессы worker; Celery не является обязательной основой любого планировщика Airflow.

В рассматриваемой конфигурации PostgreSQL хранит метаданные Airflow, Redis обслуживает очередь Celery, webserver предоставляет веб-интерфейс, scheduler планирует задачи, worker выполняет задачи, triggerer обслуживает отложенные задачи. Сервис airflow-init выполняет первоначальную подготовку базы и пользователя. Docker Compose описывает сервисы, их зависимости, переменные среды и подключённые каталоги.

3 Конфигурация проекта

Для воспроизведения задания используется версия Airflow 2.9.2. В проекте предусмотрены каталоги dags, logs, plugins, config и data. Каталог data подключается к /opt/airflow/data. Загрузка демонстрационных DAG отключается параметром AIRFLOW__CORE__LOAD_EXAMPLES=false. Для локального доступа к PostgreSQL публикуется порт 5433 компьютера в порт 5432 контейнера. В .env задаётся AIRFLOW_UID. Конфигурация проверяется командой docker compose config --quiet.

[МЕСТО ДЛЯ СОБСТВЕННОГО СКРИНШОТА 1]

Рисунок 1 — Структура проекта: docker-compose.yaml, .env и пять каталогов

[МЕСТО ДЛЯ СОБСТВЕННОГО СКРИНШОТА 2]

Рисунок 2 — Настройки LOAD_EXAMPLES, подключения data, порта PostgreSQL и подготовки каталога data

4 Инициализация и запуск сервисов

Первоначальная подготовка выполняется командой docker compose up airflow-init. Критерием успешной инициализации является завершение airflow-init с кодом 0. После этого сервисы запускаются командой docker compose up -d. Их состояние проверяется командами docker compose ps и docker compose ps -a. Остановленный airflow-init после успешного выполнения является ожидаемым состоянием, а не аварией.

[МЕСТО ДЛЯ СОБСТВЕННОГО СКРИНШОТА 3]

Рисунок 3 — Успешное завершение airflow-init с кодом 0

[МЕСТО ДЛЯ СОБСТВЕННОГО СКРИНШОТА 4]

Рисунок 4 — Состояние сервисов Airflow после запуска

5 Доступ к веб-интерфейсу

Веб-интерфейс доступен по адресу http://localhost:8080. Для учебной конфигурации методички используется пользователь airflow с паролем airflow. При отключённых примерах и отсутствии собственных файлов DAG список рабочих процессов может быть пустым. Это не свидетельствует о неисправности сервиса; создание и запуск собственного DAG в данном задании не требуются.

[МЕСТО ДЛЯ СОБСТВЕННОГО СКРИНШОТА 5]

Рисунок 5 — Веб-интерфейс Airflow после входа: список DAG без демонстрационных примеров

6 Трудности и результат проверки

[После выполнения укажите фактические трудности, предпринятые действия и результат повторной проверки. Если ошибок не возникло, укажите это. Сбои предыдущей работы с Docker не являются автоматически сбоями развёртывания Airflow.]

7 Вывод

Развёртывание Airflow демонстрирует, как Docker Compose объединяет несколько взаимосвязанных сервисов в одно приложение. Для его работы недостаточно открыть веб-страницу: необходимы база метаданных, очередь сообщений и процессы планирования и выполнения задач. Отдельная инициализация подготавливает инфраструктуру до запуска основных компонентов, поэтому её успешное завершение проверяется отдельно от состояния постоянно работающих сервисов.

Подключение каталогов отделяет файлы проекта от жизненного цикла контейнеров, а публикация портов обеспечивает доступ к веб-интерфейсу и PostgreSQL с персонального компьютера. При диагностике важно различать конфликт порта, нехватку ресурсов, неуспешную миграцию базы и ошибки доступа к файлам. Практический результат развёртывания подтверждается кодом завершения airflow-init, состоянием сервисов и доступностью интерфейса после авторизации.

[После выполнения дополните вывод фактическим результатом и тем, как были устранены возникшие трудности, как требует методичка.]

Список использованных источников

1. Знакомство с функционалом сервиса Apache Airflow : методические указания к практической работе. 12 с.

2. Apache Airflow 2.9.2. Running Airflow in Docker. URL: https://airflow.apache.org/docs/apache-airflow/2.9.2/howto/docker-compose/index.html.

3. Docker Compose. URL: https://docs.docker.com/compose/.